owis_find_deal_engine

Service guide — description, architecture, Docker, flow, cache, API clients, cost, code map, how to run, use and test it, and what comes next

Status	Phases 1–2 done · phase 3 step 11 (API clients and keys) done · next: step 12
Code	https://github.com/owisdev/claude-ai-clould-repo.git folder `0wis_Find_Deal_Engine/`, branch `claude/cloud-vs-local-7xi988`
Stack	Go 1.26 · Redis 7 · Postgres 17 · SearXNG · React 19 admin app · Docker Compose
Sources	SerpApi (Amazon, eBay, Google Shopping) · Apify (AliExpress, Temu, SHEIN) · SearXNG
Countries	usa · ksa · jor
Date	16 October 2026

How to read this guide. Sections 1–3 explain what the service is and how it is built (architecture, Docker). Sections 4–10 explain how it works: request flow, sources, cache, login and API keys, cost, settings, admin panel. Sections 11–12 map the code and the database. Sections 13–17 are hands-on: how to use the API, how to run it on your PC step by step (git and docker commands), how to test it, git workflow and troubleshooting. Sections 18–20 list the known limits, what is remaining and what is recommended next.

Contents

1. About the service 2
2. Architecture 3
3. Docker structure 4
4. Request flow (search) 7
5. Search sources per shop 9
6. Cache — how it works 10
7. Login, quotas and API clients 12
8. Cost — how it is measured 13
9. Settings 14
10. Admin panel 15
11. Main code sections 15
12. Database (Postgres) 17
13. How to use the API 18
14. Run it locally — step by step 18
15. How to test 20
16. Git workflow 22
17. Troubleshooting 22
18. Known limits (accepted for now) 23
19. What is remaining 23
20. Recommended next 23

1. About the service

owis_find_deal_engine is a backend service that receives a **product name** and a **country**, searches the online shops that **deliver to that country**, and returns **one merged list** of products with title, price, image and a direct link to the shop's product page.

It is the engine behind a future mobile / web app ("find the best deal") and can also be called by your own servers and AI agents with an API key. The goal is to keep the running cost as close to **$0** as possible: answers are cached for 48 hours, so a popular search is paid only once.

1.1 Countries and shops

Shop	usa	ksa	jor	Source (today)
Amazon	amazon.com	amazon.sa	amazon.com	SerpApi Amazon engine
eBay	ebay.com	ebay.com	— (no delivery)	SerpApi eBay engine
AliExpress	aliexpress.com	aliexpress.com	aliexpress.com	Apify scraper
Temu	temu.com	temu.com	temu.com	Apify scraper
SHEIN	us.shein.com	us.shein.com*	us.shein.com*	Apify scraper

* the SHEIN scraper reads the US site only. The country → shop list is configuration (`markets.json` or the admin panel page Countries & shops), not code.

1.2 Who calls it

Caller	Credential	Limits	Status
Mobile / web app users	Firebase login token (JWT)	plan quota per day (free 20, pro 500), 1 req/s burst 5	service ready; app not built yet
Own servers and AI agents	API key `ofd_live_…`	own quota (default 100 live searches/day), own rate	✓ step 11
Admins	Firebase login + admin list	per-admin rate limit, audit log	✓ steps 8–10
Partners (later)	API key, self-service	billing per client	later

1.3 Main features today

- One search = all shops of the country, in parallel; each shop has its own best source with fallbacks.
- Only real product pages are returned (store / category / search pages are filtered out), duplicates removed, results must match the query, at most 10 per shop.
- 48 h cache with stale-while-revalidate (72 h), pull-to-refresh, stale-if-error; **cached answers are free** for the caller.
- Login with any JWT provider (Firebase now); daily quota per plan in Redis (402 / 429); burst rate limit.
- API clients and keys for servers and agents (hashed keys, scopes, quotas, IP / country limits, rotate, revoke).
- Admin panel at `/admin/`: dashboard (usage + estimated cost + $5 budget), all settings changed without restart, test search, cache tools, admins, API clients, audit log.
- Hardened Docker stack: 4 containers, private networks, read-only, non-root, only the API port published on 127.0.0.1.

1.4 Status at a glance

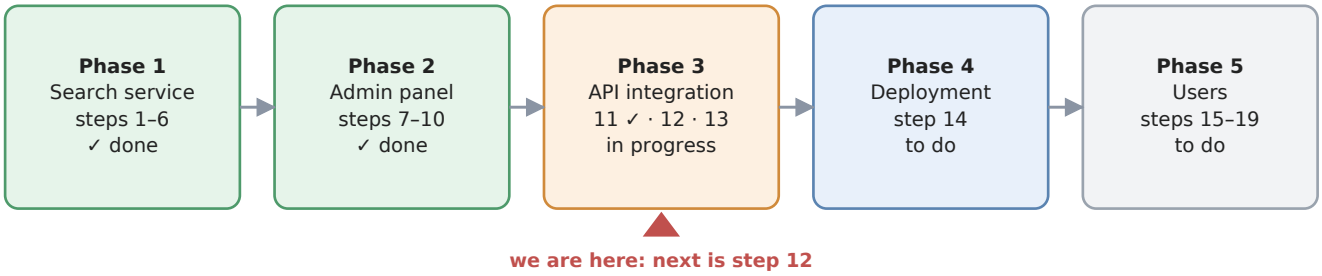


Figure 1 — Project phases. Phase 3 (API integration) is in progress.

2. Architecture

The whole service is **one Go binary** (API + admin API + admin web app) plus three helper containers. Everything runs with Docker Compose on one host today; the same image can be deployed anywhere later (phase 4).

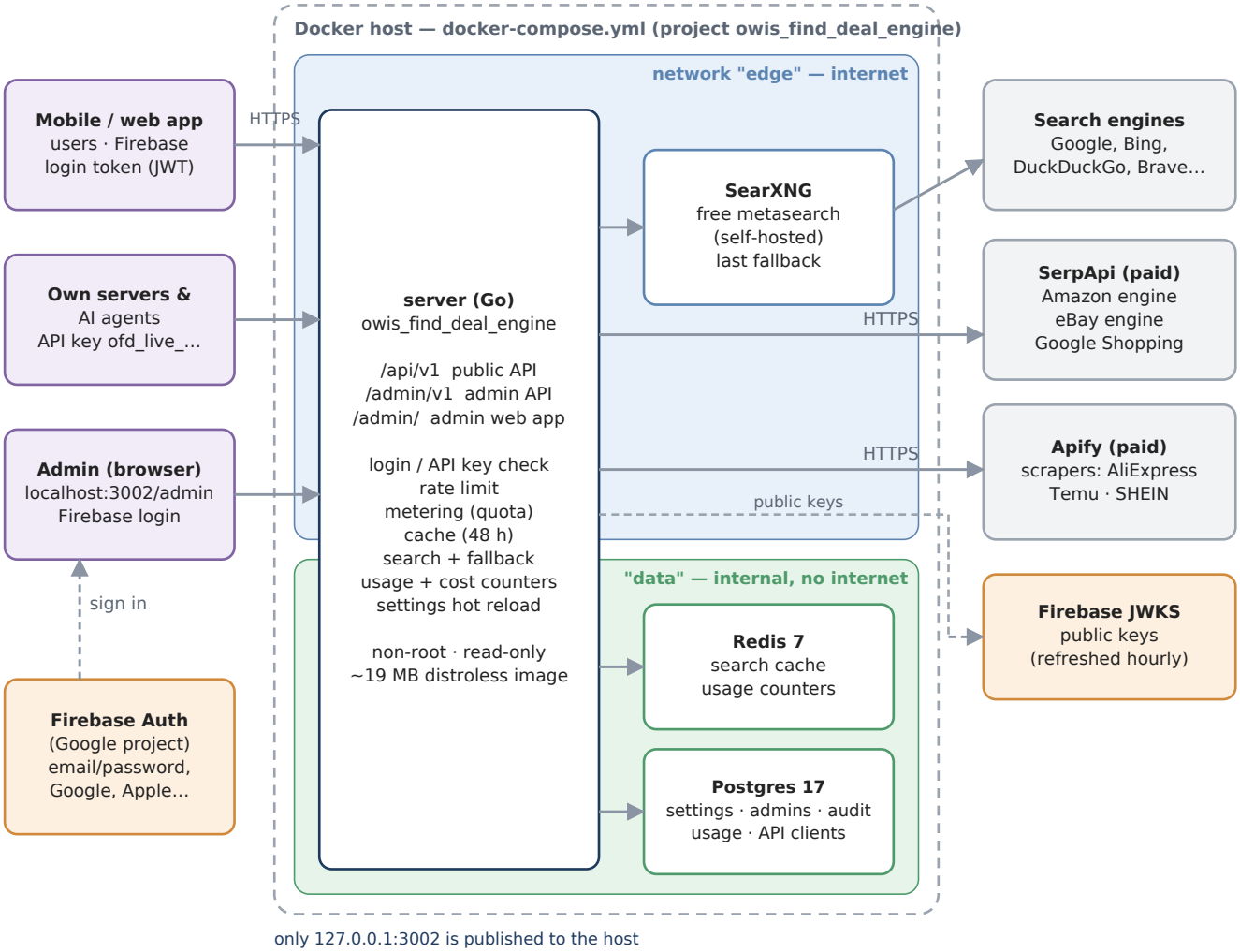


Figure 2 — Architecture. Callers on the left, the Docker stack in the middle, paid / external services on the right.

2.1 Components

Component	Role	Why
server (Go)	public API, admin API, admin web app, all business logic	one small, fast, static binary; easy to deploy
Redis	search cache (shared) + daily usage counters	atomic counters, TTL, survives restarts (AOF)
Postgres	runtime settings + history, admins, audit log, usage / cost per day, API clients + keys	changes without restart, history, durable
SearXNG	free self-hosted metasearch (last fallback)	$0 links when paid sources fail
Firebase Auth	user and admin login; the server only checks tokens with public keys (JWKS)	no passwords stored by us
SerpApi	Amazon and eBay engines (prices, direct links), Google Shopping fallback	250 free searches / month
Apify	store scrapers for AliExpress, Temu, SHEIN (no official API)	$5 free credit / month

2.2 Inside the server: layers

A request passes these layers in order. Each layer is its own Go package, so it can be tested and replaced alone:

1. **HTTP middleware** — request id, JSON log line, panic recovery, CORS allow-list, write deadline.
2. **Access** — JWT (user) or API key (client); scopes, IP and country rules for keys.
3. **Rate limit** — token bucket per user / client.
4. **Handler** — strict JSON decoding (body limit), maps results and errors to status codes + headers.
5. **Metering** — the single place that decides: cached = free, live = 1 unit of the daily quota.
6. **Cache** — fresh / stale / miss, merges identical searches, background refresh.
7. **Search service** — picks the country's shops, runs them on a bounded worker pool, merges and ranks.
8. **Provider chains** — per shop: own source → Google Shopping → SearXNG, with circuit breakers.

3. Docker structure

Files in `Owis_Find_Deal_Engine/server/`: `Dockerfile` (the image), `docker-compose.yml` (the full stack), `compose.dev.yml` (override for running Go on your PC), `.dockerignore` (keeps `.env` and tests out of the image), `deploy/searxng/settings.yml`.

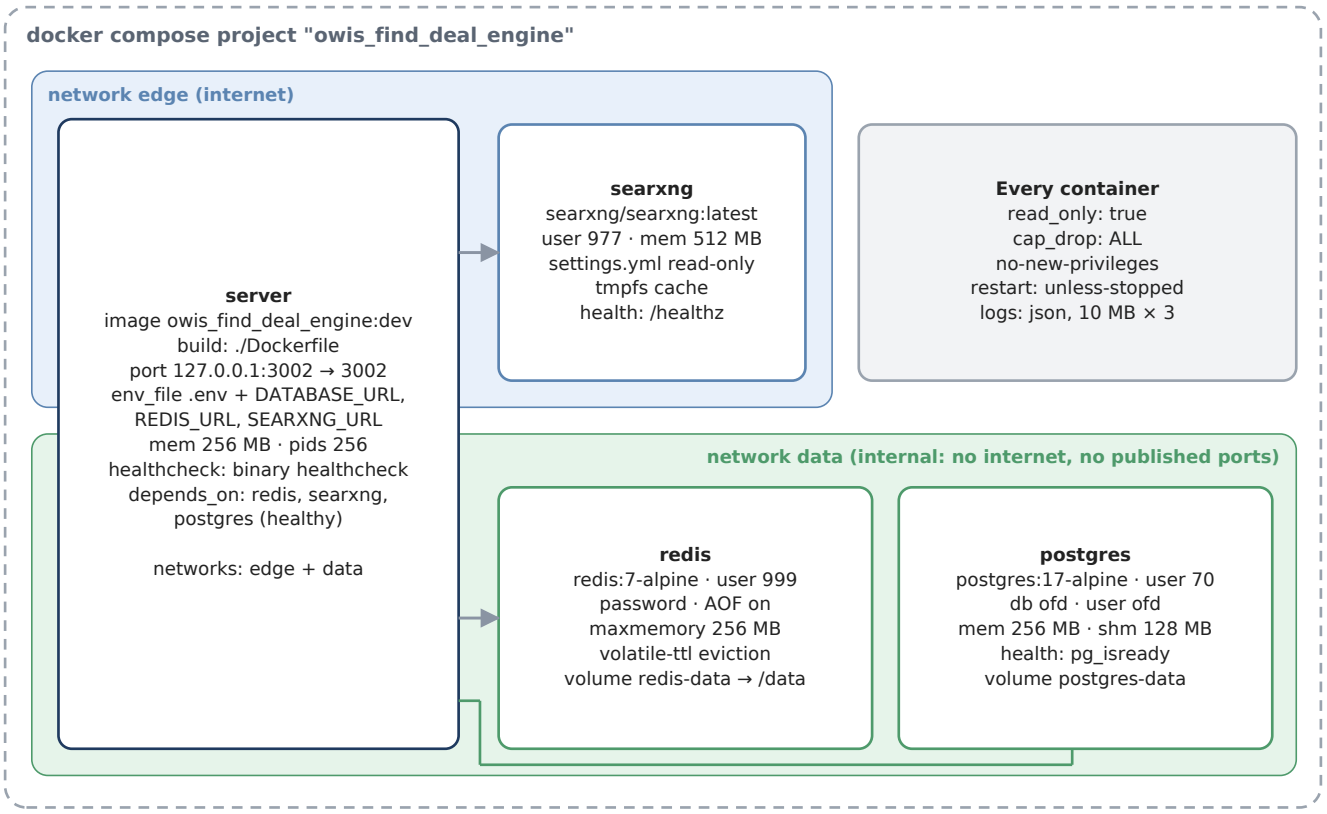
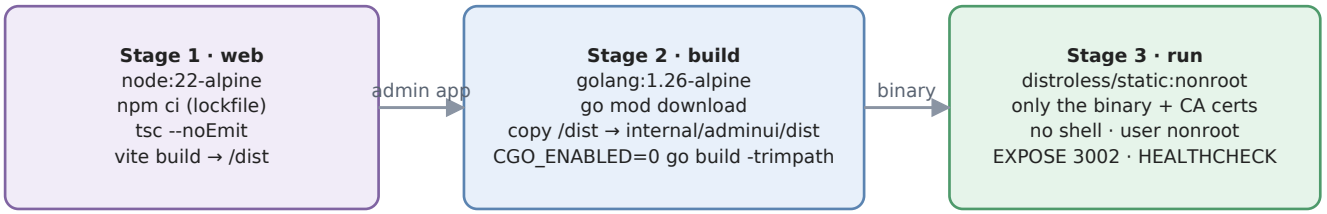


Figure 3 — Containers, networks and volumes of docker-compose.yml.

3.1 The image (multi-stage Dockerfile)



The admin web app is built and embedded in the Go binary, so one image = API + admin panel.

Figure 4 — Three build stages; the final image is ~19 MB.

3.2 Services

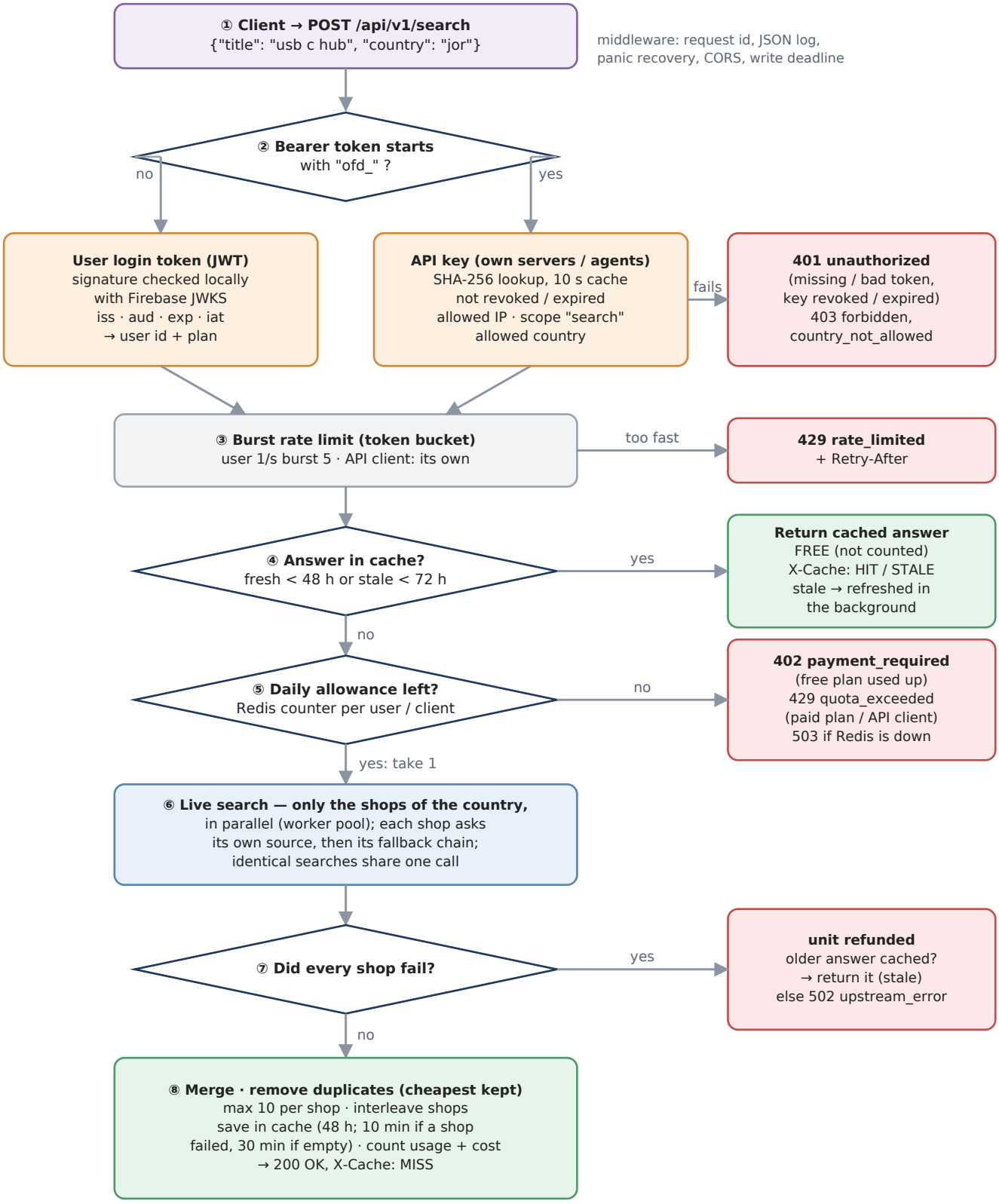
Service	Image	Networks	Ports	Data	Limits
server	built here	edge + data	127.0.0.1:3002	—	256 MB, 256 pids
searxng	searxng/searxng:latest	edge	none (dev: 127.0.0.1:8888)	tmpfs	512 MB
redis	redis:7-alpine	data	none (dev: 127.0.0.1:6379)	redis-data	320 MB (256 MB data)
postgres	postgres:17-alpine	data	none (dev: 127.0.0.1:5432)	postgres-data	256 MB

3.3 Security choices

- **Network** `data` **is internal**: Redis and Postgres have no internet and no published port. Only the server talks to them.
- **Only the API is published**, and only on `127.0.0.1` (`API_BIND`/`API_PORT`). In production put Caddy / Nginx or a Cloudflare Tunnel in front for HTTPS.
- Every container: read-only file system, all Linux capabilities dropped, no-new-privileges, non-root user, memory and process limits, log rotation.
- Passwords come from `.env`; compose refuses to start if `REDIS_PASSWORD`, `POSTGRES_PASSWORD`, `SEARXNG_SECRET` or `SETTINGS_ENCRYPTION_KEY` is missing.
- Health checks: the server image has its own `healthcheck` sub-command (distroless has no curl); the server starts only after Redis, SearXNG and Postgres are healthy.

4. Request flow (search)

What happens when a client calls `POST /api/v1/search`. Green = answer, red = refused, orange = credential checks.



Every answer carries X-Plan, X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset and, per shop, markets: ok / no_results / error.
Input check (title length, known country) happens before ④: 400 invalid_request / unsupported_country.

Figure 5 — Flowchart of one search request.

4.1 Step by step

1. **Request arrives** with `Authorization: Bearer …` and a JSON body `{"title", "country", "refresh"?}`.
2. **Credential**: a token starting with `ofd_` is an API key, anything else a Firebase JWT. JWT: signature checked locally with Firebase public keys (refreshed hourly), issuer, audience, expiry; the plan comes from a custom claim (default `free`). Key: looked up by its SHA-256 hash, must be active, allowed from this IP, have scope `search` and allow the country.
3. **Burst rate limit**: users 1 request/s with bursts of 5; each API client its own rate.
4. **Cache lookup**: a fresh or stale answer is returned immediately and is **free** — even when the daily allowance is used up.
5. **Quota**: otherwise one unit is taken from the caller's daily counter in Redis. Used up → 402 (free plan, show upgrade) or 429 (paid plan / API client). Redis down → 503 (fail closed, protects paid sources).
6. **Live search**: only the shops of the country, in parallel; each shop through its source chain (section 5). Identical concurrent searches share one upstream call.
7. **Failure handling**: if every shop fails, the unit is refunded and an older cached answer is returned if one exists (`stale: true`), else 502. If only some shops fail, the request still succeeds and `markets` shows which.
8. **Answer**: merged, deduplicated (cheapest kept), max 10 per shop, shops interleaved; saved in the cache; usage and estimated cost recorded; 200 with `X-Cache: MISS`.

4.2 Response

```
HTTP/1.1 200 OK
X-Cache: MISS            X-Plan: free          X-RateLimit-Limit: 20
X-RateLimit-Remaining: 19                      X-RateLimit-Reset: 1792195200

{
  "query": "usb c hub", "country": "jor",
  "results": [
    {"market": "amazon", "title": "Anker USB C Hub 7-in-1 ...",
     "link": "https://www.amazon.com/dp/B0...", "price": 29.99, "currency": "$",
     "thumbnail": "https://...", "position": 1, "provider": "serpapi"},
    {"market": "temu", "title": "...", "link": "https://www.temu.com/goods.html?goods_id=...",
     "price": 6.49, "currency": "$", "position": 1, "provider": "apify"}
  ],
  "markets": {"amazon": "ok", "aliexpress": "ok", "temu": "ok", "shein": "no_results"},
  "took_ms": 28650, "fetched_at": "2026-10-16T09:12:00Z", "cached": false, "stale": false
}
```

`link_type: "search"` marks a link that opens the shop's search for the product (Google Shopping gives no direct link). The app should show prices as approximate and "prices updated X ago" from `fetched_at`.

4.3 Errors

All errors look like `{"error": {"code": "…", "message": "…"}}`.

Status	code	Meaning / what to do
400	invalid_request, unsupported_country	bad JSON, title too short / long, unknown country
401	unauthorized	missing / invalid / expired JWT; unknown, revoked or expired API key
402	payment_required	free plan's daily searches used up → show upgrade
403	forbidden, country_not_allowed	API key lacks scope, wrong IP, or country not allowed
429	rate_limited, quota_exceeded	too fast (see Retry-After) / paid plan or API client used up
502	upstream_error	every shop failed and nothing cached
503	service_unavailable	Redis (usage counters) unreachable — fails closed
500	internal_error	a bug — check the server log with the request id

5. Search sources per shop

Free web search alone was not enough (blocked quickly from one IP, finds few product pages, no prices). So each shop has its own best source, chosen in the October 2026 local tests:

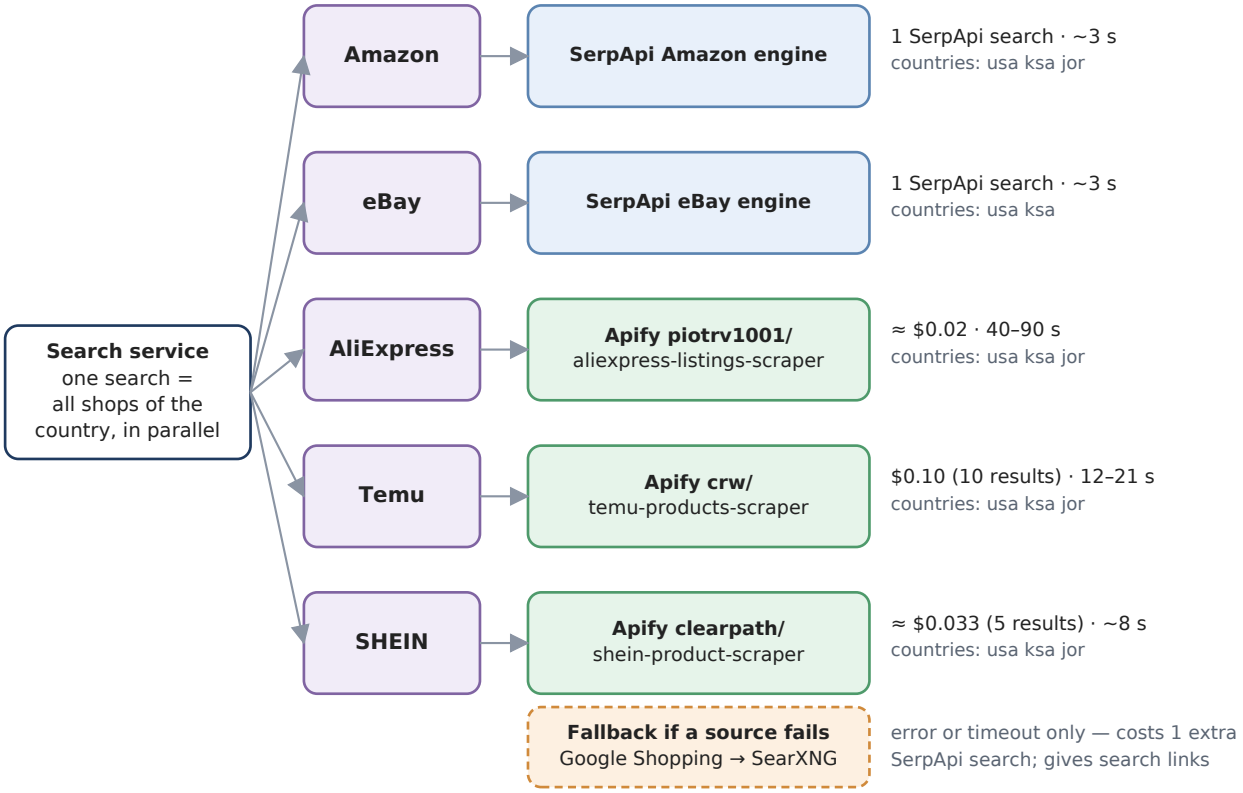
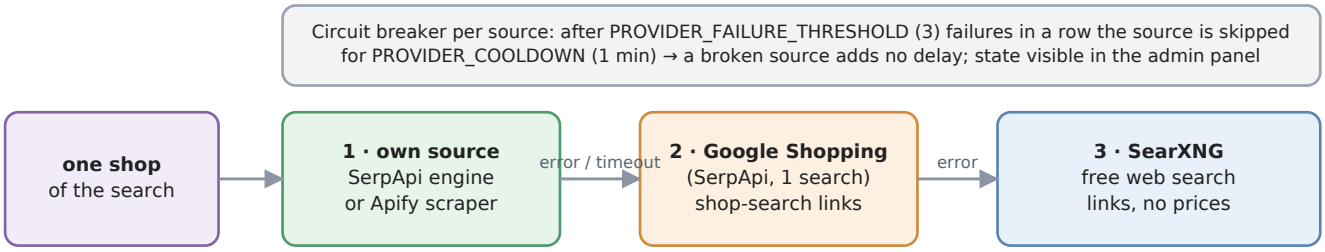


Figure 6 — Source, cost and speed per shop (October 2026 measurements).

5.1 Fallback chain and circuit breaker



Each attempt has its own timeout: PROVIDER_ATTEMPT_TIMEOUT 8 s, SERPAPI_TIMEOUT 25 s, APIFY_TIMEOUT 60–90 s.
APIFY_EMPTY_FALLBACK=false: a scraper that finds nothing relevant ends as no_results (no extra cost).
The first source that answers wins; every product carries "provider": serpapi / apify / searxng.

Figure 7 — What happens when a shop's own source fails.

5.2 Result quality rules

- **Product pages only**: each shop has `product_pages` patterns (Amazon `/dp/<ASIN>`, eBay `/itm/<id>`, AliExpress `/item/<id>.html`, Temu `goods.html?goods_id=`, SHEIN `-p-<id>.html`).
- **Relevance**: at least two thirds of the query words must appear in the title, snippet or link.
- **Clean links**: tracking parameters removed, the same product listed once (cheapest kept).
- **Per shop limit**: `RESULTS_PER_MARKET` = 10, best first, shops interleaved.

5.3 Typical timings

Case	Time	Notes
Cached answer (HIT / STALE)	< 50 ms	free
Live, SerpApi shops only (Amazon, eBay)	~3 s	1–2 SerpApi searches
Live, all shops incl. Apify scrapers	25–50 s	scrapers start a browser; run in parallel

6. Cache — how it works

The cache is the main cost saver: a live search may cost ~$0.16–0.19, a cached answer costs nothing. It sits in front of the search service (`internal/cache`) and is stored in Redis (or in memory when `REDIS_URL` is empty).



A shop failed → that answer is fresh only 10 min · no results → 30 min · pull-to-refresh at most once per 10 min per search.

Figure 8 — Life of a cached answer (default settings).

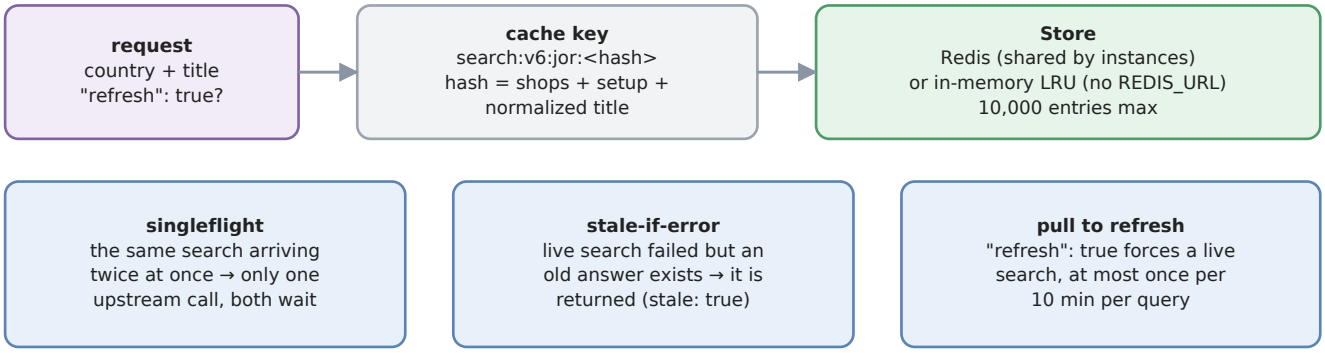


Figure 9 — Cache key, storage and the three safety features.

6.1 Rules

Age / situation	What happens	X-Cache	Setting
< 48 h	served from cache, free	HIT	CACHE_FRESH_TTL
48 – 72 h	served instantly (stale: true) and refreshed in the background	STALE	CACHE_STALE_TTL
> 72 h or not cached	live search	MISS	—
a shop failed	answer fresh only 10 min (retried sooner)		CACHE_PARTIAL_TTL
no results at all	fresh only 30 min		CACHE_EMPTY_TTL
"refresh": true	forces live search, at most once per 10 min per query	MISS	CACHE_MIN_REFRESH
background refreshes	at most 4 in parallel		CACHE_MAX_BACKGROUND_REFRESH

6.2 The cache key

```
search:<results version>:<country>:<first 16 bytes of SHA-256(
        shop-list version ; search setup ; shop=domain;... ; normalized title )>
example: search:v6:jor:3f9a0c...    (title: lower case, spaces collapsed)
```

- **Same search, any spelling of spaces / case** → same key, so users share answers.
- **Shop list or search setup changed** (markets, provider order, results per shop) → new key; old answers are never served for a changed setup.
- **Results version** (v6 in code) is raised when result parsing changes, so a new build drops answers from an older one.
- **Admin tools**: forget one search (next search is live) and drop all cached answers (raises `CACHE_GENERATION`).
- **Redis memory**: max 256 MB, policy `volatile-ttl` — cache entries (closest to expiry) are evicted before usage counters.

7. Login, quotas and API clients

7.1 App users (Firebase JWT)

- The app signs the user in with Firebase (email, Google, Apple…) and sends the ID token as `Authorization: Bearer <JWT>`.
- The server checks it **locally** with Firebase's public keys (JWKS, cached, refreshed hourly; RS256 / ES256 only, `alg: none` and HMAC refused) — no call to Firebase per request.
- Plan from the custom claim `plan` (`AUTH_PLAN_CLAIM`); plans in `PLANS=free:20,pro:500` (live searches per UTC day).

7.2 API clients (own servers and AI agents)

Owner decision D4: keys are for **servers and agents only**. The mobile / web app keeps the user's login token, because a key inside an app could be extracted.

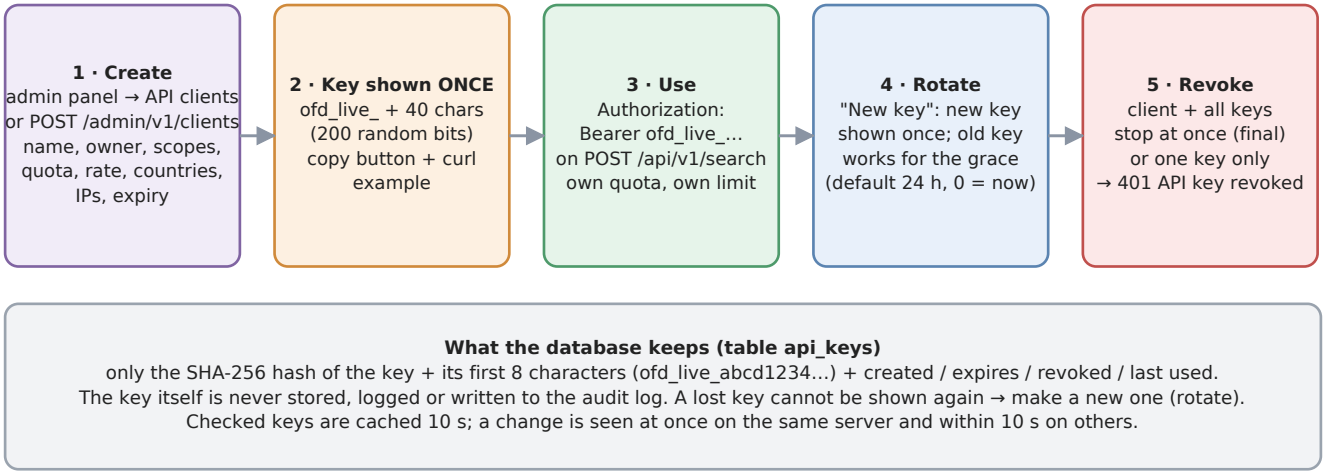


Figure 10 — Life of an API key.

Client setting	Default	Effect
name, owner	—	who uses it (shown in panel and audit log)
scopes	search, countries	search = may call /api/v1/search; countries = for agent tools (step 13)
daily_quota	100	live searches per UTC day (cached answers free); used up → 429 quota_exceeded
rate_rps / rate_burst	1 / 5	burst limit of this client only
countries	all	other country → 403 country_not_allowed
ip_allow	any	list of IPs / CIDRs; other address → 403 forbidden
expires_at	never	the client and all its keys stop at that time

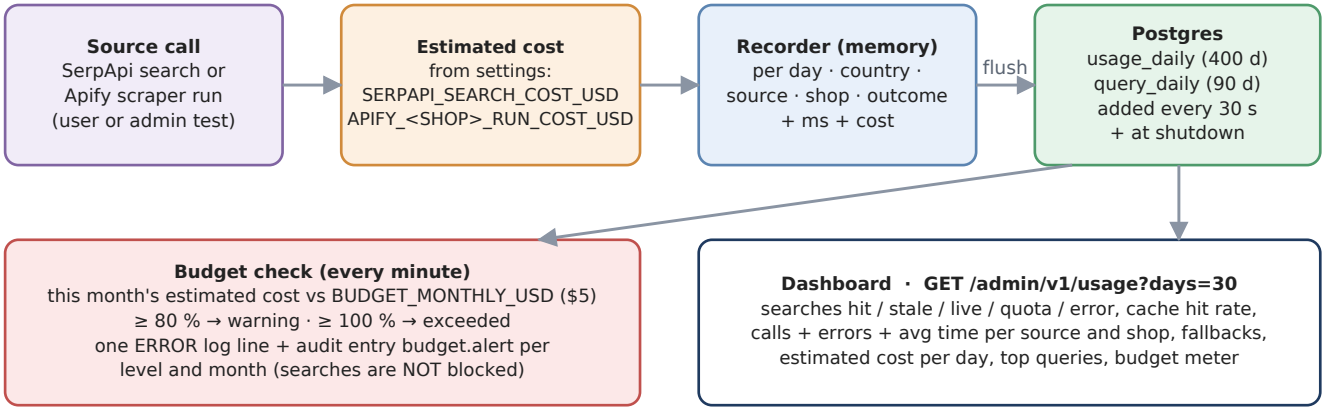
> Counters never mix: a user's counter is the user id, a client's is `client:<id>`, and a login token whose subject starts with `client:` is refused. Answers to API clients carry `X-Plan: api`.

7.3 Admins

- Admin API `/admin/v1/…` and panel `/admin/`: valid Firebase token + active entry in the `admins` table + sign-in not older than 12 h.
- The first admin comes from `ADMIN_EMAIL`; more with the panel or `admins add EMAIL`.
- **Disabled for development (must be turned on before production)**: two-factor login (`ADMIN_REQUIRE_2FA`) and email verification (`ADMIN_REQUIRE_VERIFIED_EMAIL`).
- Every change, test search, cache tool and refused login goes to the audit log (never secret values).

8. Cost — how it is measured

The service pays only for **live** searches that reach a paid source. It cannot read the providers' bills, so it **estimates** the cost of every call from price settings and adds it up per day.



Costs are ESTIMATES (price per call × number of calls). The real bill is in the SerpApi and Apify consoles — compare once a month and adjust the cost settings. Cached answers cost nothing and are not counted.

A source call counts once per shop for Apify (one run per shop) and once per call for SerpApi.

Figure 11 — From a source call to the dashboard and the budget alert.

8.1 The formula

```
cost of one live search = Σ over shops that called a paid source:
     SerpApi call  → SERPAPI_SEARCH_COST_USD          (per search, any number of shops)
     Apify run     → APIFY_<SHOP>_RUN_COST_USD        (per scraper run = per shop)
     SearXNG       → 0
cost of a cached answer = 0
month cost = Σ usage_daily.cost_usd for this month (all servers add up)
```

8.2 Current prices (owner's accounts, October 2026)

Source	Measured price	Setting to enter
Temu (Apify crw/temu-products-scraper)	$0.10 per run (charged for 10 results)	APIFY_TEMU_RUN_COST_USD = 0.10
SHEIN (Apify clearpath/shein-product-scraper)	≈ $0.0066 per result, 5 results	APIFY_SHEIN_RUN_COST_USD = 0.033
AliExpress (Apify piotrv1001/…)	≈ $0.02 per run	APIFY_ALIEXPRESS_RUN_COST_USD = 0.02
SerpApi (Amazon, eBay, Shopping)	free plan: 250 / month · Developer ≈ $0.015	SERPAPI_SEARCH_COST_USD = 0 (free) or 0.015

Worked example — live search in `jor`: Temu 0.10 + SHEIN 0.033 + AliExpress 0.02 = **$0.153 Apify** + 1 SerpApi search (Amazon; eBay does not deliver to Jordan). In usa / ksa: same Apify + 2 SerpApi searches (Amazon + eBay). Repeating it within 48 h costs **$0**. With the $5 Apify free credit that is ≈ 32 different live searches per month.

8.3 Budget

- `BUDGET_MONTHLY_USD` = **5** (owner decision D5: free plans), `BUDGET_ALERT_PERCENT` = 80.
- Status on the dashboard: `none / ok / warning / exceeded`. At 80 % and 100 %: one ERROR log line and one audit entry `budget.alert` per level and month.
- Searches are **not** blocked when the budget is exceeded (a hard stop can be added — see section 20).
- **Important:** if the cost settings are not entered, every cost shows 0. Enter them once in Settings (or with the CLI) and compare with the Apify / SerpApi consoles monthly.

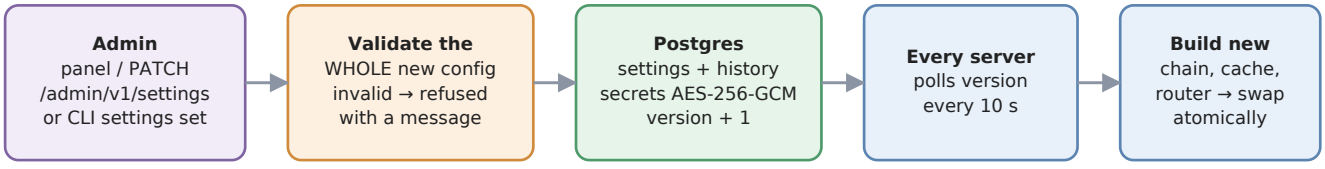
8.4 What is counted (table usage_daily)

kind	keys	values
search	country, outcome = hit / stale / live / quota / error	count
source	provider, shop, outcome = ok / no_results / error	count, cost_usd, total_ms
fallback	chain, from, to : reason (error / partial / circuit_open)	count
query_daily	country, normalized query (no user ids)	searches, live

9. Settings

Two kinds of settings (`internal/config/keys.go`):

Kind	Where	Examples	Change
bootstrap	.env only	DATABASE_URL, SETTINGS_ENCRYPTION_KEY, AUTH_*, ADMIN_EMAIL, REDIS_URL, PORT, LOG_LEVEL	edit .env + restart
runtime	Postgres (copied from .env on first start)	MARKETS, SEARCH_PROVIDERS, APIFY_*, SERPAPI_*, CACHE_*, PLANS, RATE_LIMIT_*, BUDGET_*, costs	admin panel / CLI, no restart



No restart. Searches already running finish on the old setup. .env is copied into the database only on the FIRST start; afterwards change runtime settings in the panel / CLI (the server warns about .env keys that differ). Secrets are write-only: never shown again, never in the history. Bootstrap settings (database, encryption key, auth, ports) stay in .env and need a restart.

Figure 12 — A settings change is applied on every server within ~10 s.

9.1 Command line inside the container

```
export MSYS_NO_PATHCONV=1                 # Git Bash on Windows only
S="docker compose exec server /owis_find_deal_engine"
$S settings list                          # value or (default), who changed it
$S settings set APIFY_SHEIN_MAX_ITEMS 5 CACHE_FRESH_TTL 48h
docker compose exec -T server /owis_find_deal_engine settings set SERPAPI_KEY - < serpapi.key
$S settings unset CACHE_FRESH_TTL         # back to the default
$S settings history [KEY]
$S admins list | add EMAIL | disable EMAIL | enable EMAIL | unlink EMAIL
$S audit [ACTION-PREFIX]                  # latest admin actions
$S clients list                           # API clients, key prefixes, last use
$S clients revoke ID                      # emergency stop of a client and all its keys
```

10. Admin panel

http://localhost:3002/admin/ — React + TypeScript app built into the server image, same origin as the admin API, strict Content-Security-Policy, only `@firebase/app` + `@firebase/auth`. Sign in with the Firebase email + password of an admin; the session ends when the tab closes.

Page	What you do there
Dashboard	tiles, searches per day (cached vs live), estimated cost per day, budget meter, sources per shop, fallbacks, countries, top queries
Settings	every runtime setting grouped with help text; secrets write-only; changes saved together and checked as a whole; change history
Countries & shops	overview + JSON editor of markets (validated, applied without restart)
Test search	asks each shop's source directly (no cache / quota / fallback): products, time, error, estimated cost; optional all sources
Cache & sources	server status, forget one search, drop all cached answers, circuit breaker state
API clients	create, edit, new key with grace period, stop old key, revoke; key shown once + curl example
Admins	add, disable, enable, unlink (never yourself or the last admin)
Audit log	who did what, when, from which IP, with request id

11. Main code sections

Go module `owis_find_deal_engine` in `0wis_Find_Deal_Engine/server/`. About 16,000 lines of Go incl. tests, 188 test functions. Each row is a package and what it is related to:



Figure 13 — Code map, from the entry point down to the data layer.

11.1 Key files and what they relate to

File	Related to	What to know
cmd/…/main.go	everything	run(): config → Postgres + migrations → JWKS → Redis → buildApp() → settings watcher (atomic swap) → HTTP server → graceful shutdown. sourceCost()/callCost(): cost estimate per call
cmd/…/admincmd.go	settings, admins, audit, clients	the CLI sub-commands
internal/api/router.go	public API	routes + middleware order; access.go: JWT or key; handlers.go: search handler
internal/api/admin*.go	admin API	admin access rules, settings, markets, tools, usage, clients
internal/metering/metering.go	quota + cost rule	cached = free; take / refund units; reports outcome
internal/cache/cache.go	cache	Lookup / Search, key(), resultsVersion = v6, background refresh
internal/search/search.go	search	worker pool, merge, dedupe, limitPerMarket
internal/search/fallback.go	fallbacks	chain + circuit breaker; probe.go = admin test search
internal/providers/*	sources	serpapi.go, amazon.go, ebay.go, apify.go, searxng.go
internal/markets/markets.json	countries / shops	domains, product page patterns, shopping region
internal/apikeys/*.go	API clients	Generate(), Hash(), Authenticate() with 10 s cache, client rules
internal/stats/*.go	cost dashboard	Recorder (memory) → Flush (30 s) → BuildReport, CheckBudget
internal/store/migrations	database	0001 settings + admins, 0002 audit, 0003 usage, 0004 API clients
web/admin/src	admin web app	App.tsx, api.ts, auth.ts, pages/*.tsx

12. Database (Postgres)

Table	Contents	Since
settings / settings_history	runtime settings (secrets encrypted) and every change: who, when, old → new	step 7
admins	email, linked Firebase uid, disabled	step 7
audit_log	admin actions, test searches, refused logins, budget alerts	step 8
usage_daily / query_daily	counters per day and kind (400 days) / top queries (90 days)	step 9
api_clients / api_keys	client settings / key hash, prefix, expiry, revoked, last used	step 11
schema_migrations	applied migrations (run automatically at start, advisory lock)	step 7

13. How to use the API

Method	Path	Auth	Purpose
GET	/api/v1/health	—	liveness
GET	/api/v1/countries	—	countries and their shops
POST	/api/v1/search	JWT or API key	search; body {"title", "country", "refresh"?}
*	/admin/v1/...	admin JWT	settings, markets, admins, audit, tools, usage, clients
GET	/admin/	browser	admin web app

13.1 Examples (Git Bash / macOS / Linux)

```
curl localhost:3002/api/v1/health
curl localhost:3002/api/v1/countries

# as an app user (Firebase ID token, valid 1 hour)
curl -i -X POST localhost:3002/api/v1/search \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"title":"usb c hub","country":"jor"}'

# pull to refresh (live again, at most once per 10 min)
  -d '{"title":"usb c hub","country":"jor","refresh":true}'

# as an API client (own server / AI agent)
curl -i -X POST localhost:3002/api/v1/search \
  -H "Authorization: Bearer ofd_live_xxxxxxxx..." -H "Content-Type: application/json" \
  -d '{"title":"usb c hub","country":"jor"}'

# admin: usage and cost of the last 7 days
curl -H "Authorization: Bearer $ADMIN_TOKEN" "localhost:3002/admin/v1/usage?days=7"
```

13.2 PowerShell

```
curl.exe -i -X POST localhost:3002/api/v1/search `
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" `
  -d '{\"title\":\"usb c hub\",\"country\":\"jor\"}'

# or natively
Invoke-RestMethod -Method Post -Uri http://localhost:3002/api/v1/search `
  -Headers @{Authorization="Bearer $TOKEN"} -ContentType "application/json" `
  -Body '{"title":"usb c hub","country":"jor"}'
```

14. Run it locally — step by step

> **You need:** git, Docker Desktop (Windows / macOS) or Docker Engine + compose plugin (Linux), a free Firebase project. Optional: SerpApi key (prices for Amazon / eBay), Apify token (AliExpress / Temu / SHEIN). Go 1.26 and Node 22 are only needed for development without Docker.

Step 1 — Get the code (git)

```
git clone https://github.com/owisdev/claude-ai-clould-repo.git
cd claude-ai-clould-repo
git fetch origin
git checkout claude/cloud-vs-local-7xi988
git pull origin claude/cloud-vs-local-7xi988          # later: get the newest version
git log --oneline -5              # latest: "Phase 3 step 11: API clients and keys"
cd Owis_Find_Deal_Engine/server
```

Step 2 — Firebase project (login tokens)

1. `https://console.firebase.google.com` → **Add project** (free plan).
2. **Build → Authentication → Get started** → enable **Email/Password**.
3. **Project settings → General**: note the **Project ID** and the **Web API key** (add a Web app if no key is shown).
4. **Authentication → Users → Add user**: your admin login (e.g. `admin@example.com`; a made-up email works while verification is off).

Step 3 — Configure .env

```
cp .env.example .env
openssl rand -hex 32     # run 4 times: for SEARXNG_SECRET, REDIS_PASSWORD, POSTGRES_PASSWORD,
                         # SETTINGS_ENCRYPTION_KEY  (PowerShell:
                         # -join ((1..32) | % { '{0:x2}' -f (Get-Random -Max 256) }) )
```

Then set at least these lines in `.env`:

```
AUTH_JWKS_URL=https://www.googleapis.com/service_accounts/v1/jwk/securetoken@system.gserviceaccount.com
AUTH_ISSUER=https://securetoken.google.com/<PROJECT_ID>
AUTH_AUDIENCE=<PROJECT_ID>
SEARXNG_SECRET=<hex>
REDIS_PASSWORD=<hex>
POSTGRES_PASSWORD=<hex>
SETTINGS_ENCRYPTION_KEY=<hex>          # keep a copy! DB secrets unreadable without it
ADMIN_EMAIL=admin@example.com          # your Firebase admin login
ADMIN_FIREBASE_API_KEY=<WEB_API_KEY>   # for the admin panel sign-in

# optional paid sources (prices):
SERPAPI_KEY=<key>                      # https://serpapi.com (Amazon + eBay engines turn on)
SEARCH_PROVIDERS=serpapi,searxng
APIFY_TOKEN=<token>                    # https://console.apify.com/settings/integrations
APIFY_MARKETS=aliexpress,temu,shein
# + the APIFY_<SHOP>_ACTOR / _INPUT lines from .env.example (tested values are there)
```

> **Note:** runtime settings (sources, cache, plans…) are copied from `.env` into Postgres **only on the first start**. To change them later use the admin panel or `settings set`; or reset everything with `docker compose down -v` (deletes the database and cache).

Step 4 — Start the stack (docker)

```
docker compose up -d --build     # first build: a few minutes (Go + admin web app)
docker compose ps                # wait until all four are "healthy" (~30 s)
docker compose logs -f server    # look for "runtime settings imported from the environment"
```

Step 5 — Get a test token (valid 1 hour)

```
curl -s -X POST "https://identitytoolkit.googleapis.com/v1/accounts:signUp?key=<WEB_API_KEY>" \
  -H "Content-Type: application/json" \
  -d '{"email":"test1@example.com","password":"Test-pass-123","returnSecureToken":true}'
# copy "idToken" → export TOKEN=...   (next time: accounts:signInWithPassword, same body)
# admin token: accounts:signInWithPassword with the ADMIN_EMAIL user → export ADMIN_TOKEN=...
```

Step 6 — Try it

```
curl localhost:3002/api/v1/health
curl -i -X POST localhost:3002/api/v1/search -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"title":"usb c hub","country":"jor"}'
# run the same command again → X-Cache: HIT, instant, X-RateLimit-Remaining does not go down
```

Step 7 — Admin panel and an API key

1. Open **http://localhost:3002/admin/** and sign in with the admin email + password.
2. **Settings** → add the cost settings: `APIFY_TEMU_RUN_COST_USD` 0.10, `APIFY_SHEIN_RUN_COST_USD` 0.033, `APIFY_ALIEXPRESS_RUN_COST_USD` 0.02 → Save.
3. **API clients** → name `test agent`, tick `jor` → Create client and key → copy the key (shown once).
4. Search with the key (section 13) → 200 with `X-Plan: api`; with `"country":"usa"` → 403 country_not_allowed.
5. **Dashboard** after ~30 s: searches, cache hits, estimated cost, budget.

Step 8 — Stop / reset

```
docker compose stop             # stop (graceful), keep data
docker compose down             # remove containers, keep volumes (settings, cache, counters)
docker compose down -v          # ALSO delete Redis + Postgres data (next start imports .env)
```

Option — run the Go server on your PC (development)

```
docker compose -f docker-compose.yml -f compose.dev.yml up -d searxng redis postgres
# in .env:  SEARXNG_URL=http://127.0.0.1:8888
#           REDIS_URL=redis://:<REDIS_PASSWORD>@127.0.0.1:6379/0
#           DATABASE_URL=postgres://ofd:<POSTGRES_PASSWORD>@127.0.0.1:5432/ofd?sslmode=disable
set -a; . ./.env; set +a
go run ./cmd/owis_find_deal_engine

# admin web app with hot reload (another terminal)
cd web/admin && npm ci && npm run dev      # http://localhost:5173/admin/ (proxies to :3002)
```

15. How to test

15.1 Automated tests (same as CI)

```
cd Owis_Find_Deal_Engine/server
gofmt -l .                          # must print nothing
go vet ./...
go test -race ./...                 # all packages (store tests skip without a database)

# store tests against a throwaway Postgres (its tables are dropped!)
docker run -d --name ofd-test-db -e POSTGRES_USER=ofd -e POSTGRES_PASSWORD=ofd \
  -e POSTGRES_DB=ofd_test -p 127.0.0.1:55432:5432 postgres:17-alpine
TEST_DATABASE_URL=postgres://ofd:ofd@127.0.0.1:55432/ofd_test?sslmode=disable \
  go test -race -count=1 ./internal/store
docker rm -f ofd-test-db

go run golang.org/x/vuln/cmd/govulncheck@latest ./...     # known vulnerabilities

cd web/admin && npm ci && npm run build && npm audit --omit=dev   # admin app
```

Result on this branch (16 Oct 2026): **gofmt clean, go vet clean, all 17 test packages pass**. GitHub Actions (`.github/workflows/server.yml`) runs the same on every push: format, mod tidy, vet, race tests with Postgres 17, govulncheck, admin app build + audit, compose validation, image build and smoke test.

15.2 What the tests cover

Area	Tests
auth	JWT signature / issuer / audience / expiry, alg none and HMAC refused, JWKS refresh and key rotation
api	every route and error code, JWT vs API key, scopes, IP, country, quotas, admin access rules, every admin endpoint
cache	fresh / stale / miss, partial and empty TTL, refresh limit, singleflight, stale-if-error, keys
search	worker pool, merge, dedupe, relevance, fallback chain, circuit breaker, probe
providers	SerpApi (Shopping, Amazon, eBay), Apify (AliExpress, Temu, SHEIN sample outputs), SearXNG
metering / usage / ratelimit	free cached answers, take / refund, 402 vs 429, token bucket
settings / store / stats	validation, encryption, import, watcher; Postgres store; counters, report, budget
apikeys	format, hashing, cache, expiry, rotation grace, scope / IP / country rules

15.3 Testing the Docker stack

```
docker compose config -q                      # compose file valid (needs .env)
docker compose build --no-cache server        # clean image build
docker images owis_find_deal_engine           # size ~19 MB
docker compose up -d && docker compose ps     # 4 × healthy
docker inspect --format '{{.State.Health.Status}}' owis_find_deal_engine-server-1

# smoke test of the image alone (what CI does):
docker run --rm --entrypoint /owis_find_deal_engine owis_find_deal_engine:dev healthcheck
                                              # ↑ must fail: no server is running
docker run --rm owis_find_deal_engine:dev     # must refuse to start without configuration

# security checks
docker compose exec server id                 # fails as expected: distroless has no tools
docker inspect owis_find_deal_engine-server-1 \
  --format '{{.HostConfig.ReadonlyRootfs}} {{.HostConfig.CapDrop}}'      # true [ALL]
docker network inspect owis_find_deal_engine_data --format '{{.Internal}}'      # true

# logs, resources, inside services
docker compose logs --tail=100 server
docker stats --no-stream
docker compose exec redis redis-cli --scan --pattern 'search:*' | head    # cached answers
docker compose exec postgres psql -U ofd -d ofd -c \
  'select day,kind,k1,k2,count,cost_usd from usage_daily order by day desc limit 20;'
docker compose exec server /owis_find_deal_engine settings list
```

15.4 Manual test checklist

1. **Health**: `/api/v1/health` 200; 4 containers healthy.
2. **Login**: no token → 401; expired token → 401; valid token → 200.
3. **Search** in usa, ksa, jor: shops correct (no eBay in jor), links open product pages, prices shown.
4. **Cache**: same search again → `X-Cache: HIT`, remaining quota unchanged; `"refresh":true` → MISS (second time within 10 min → HIT).
5. **Quota**: set `PLANS=free:2` in Settings, 3 different searches → third gives 402.
6. **API key**: create, search (200, X-Plan api), wrong country 403, rotate with 1 h grace (both work), stop old (401 expired), revoke (401 revoked), audit log shows clients.* entries without keys.
7. **Settings hot reload**: change `APIFY_SHEIN_MAX_ITEMS` → log "settings applied" within 10 s, no restart.

8. **Cost**: after live searches, Dashboard cost ≈ Apify console (Billing → usage).
9. **Test search** (admin): each source answers with time and cost; breakers closed.

16. Git workflow

```
git status                                  # what changed
git switch claude/cloud-vs-local-7xi988             # the project branch
git pull origin claude/cloud-vs-local-7xi988        # update
git switch -c my-change                     # own branch for a change
git add -A && git commit -m "Describe the change"
git push -u origin my-change                # then open a pull request into the project branch
git log --oneline --graph -15               # history
git diff origin/claude/cloud-vs-local-7xi988         # compare with the project branch
git stash / git stash pop                   # park local edits

# never commit .env (it is in .gitignore). Check:  git check-ignore -v server/.env
```

Branches: work happens on `claude/cloud-vs-local-7xi988` (steps 7–11 merged there through pull requests #2–#6). `main` does not have the service yet — merging it into `main` is part of the next steps (section 20).

17. Troubleshooting

Problem	Cause / fix
compose: "set REDIS_PASSWORD in .env"	a required secret is empty in .env — fill all four hex values
container unhealthy	`docker compose logs server postgres redis searxng`; often a wrong AUTH_* or DATABASE_URL
401 on every search	token expired (1 h) or AUTH_ISSUER / AUTH_AUDIENCE not the Firebase project id
Git Bash: "no such file" with exec	`export MSYS_NO_PATHCONV=1` or use `//owis_find_deal_engine`
.env change has no effect	runtime settings are in Postgres after the first start — use Settings / CLI, or `down -v`
admin panel: "not an admin"	email not in the admin list → `admins add EMAIL`; ADMIN_FIREBASE_API_KEY missing → no sign-in form
a shop always error / slow	Admin → Test search; check APIFY_TIMEOUT (90 s for AliExpress), Apify credit, breaker state
Apify "maximum cost of $0.00"	set APIFY_MAX_CHARGE_USD (default 0.20) for per-event scrapers (SHEIN)
costs all 0 on the dashboard	cost settings not entered (section 8.2)
503 service_unavailable	Redis down or wrong password — quota counters fail closed
lost SETTINGS_ENCRYPTION_KEY	stored secrets unreadable → set SERPAPI_KEY / APIFY_TOKEN again

18. Known limits (accepted for now)

- **Scraper prices are US prices** (AliExpress, Temu, SHEIN read the US sites); they can differ from what a user in Jordan / KSA sees. The app should show "from $X · approx." — the link shows the real price.
- **AliExpress welcome deals**: some prices are new-shopper offers → app note.
- **SHEIN** links are `us.shein.com` in every country (scraper supports US only).
- **First live search takes 25–50 s** (scrapers start a browser) → loading bar in the app; async search is step 12.
- **Budget alert does not block** searches; costs are estimates.
- **2FA and email verification for admins are off** (development only).
- **One host, no HTTPS, no backups** yet — local development stack.
- Behind a reverse proxy the API-key IP allow-list would see the proxy address until the proxy header is trusted (step 14) — leave the IP list empty until then.

19. What is remaining

Step	What	Status
12	Integration endpoints: async search `POST /api/v1/searches` → 202 + id, `GET /api/v1/searches/{id}`, optional signed (HMAC) webhook; per-client usage and cost on the dashboard	**next**
13	AI agents: MCP server `/mcp` (tools search_products, list_countries) with API keys; OpenAPI 3.1 spec + developer docs page	to do
14	Deployment: 2FA + email verification on; hosting (D6), domain (D7), HTTPS, secrets store, Postgres backups, staging + production, CI/CD from main, monitoring + alerts, gobreaker	to do
15–19	Users: users table + /me, plans in DB + payment webhook, saved cart + price tracker, clicks + purchase reports, notifications (inbox, FCM push, background jobs)	to do
later	eBay Browse API / AliExpress affiliate API if keys are approved; cheaper or Saudi SHEIN scraper; search by photo (Google Lens)	ideas

Open owner decisions: **D6** hosting, **D7** domain (step 14), **D8** paid Apify / SerpApi plans (before real users), payment provider (phase 5).

20. Recommended next

In priority order — the first items are cheap and remove risk before more features are added:

1. **Merge into main.** Open a pull request `claude/cloud-vs-local-7xi988 → main` so the default branch holds the real service (today it does not); protect main and require the CI checks.
2. **Owner test of step 11** on the Docker stack (section 14, step 7) and enter the cost settings, so the dashboard shows real estimates.
3. **Step 12 — async search + webhook**: removes the 25–50 s wait for agents and servers; add per-client usage to the dashboard (base for billing).
4. **Optional hard budget stop**: a setting such as `BUDGET_HARD_STOP=true` that serves cached answers only once the month's budget is exceeded — protects the $5 free credit.
5. **Pre-warm popular queries** before they expire (top queries per country are already counted) — more cache hits at almost no cost.
6. **Step 13 — MCP server + OpenAPI** so AI agents (Claude, ChatGPT…) can use the service as a tool.

7. **Before any public deployment (step 14)**: turn on admin 2FA and email verification, HTTPS via Caddy or Cloudflare Tunnel, Postgres backups (daily dump + restore test), secrets out of .env files, pin the SearXNG image version (today `latest`), metrics + alerts.
8. **Choose hosting (D6)**: a ~$5/month VPS (Hetzner / DigitalOcean) runs the whole compose stack as is.
9. **Then the app and users (phase 5)**, with the UI notes: approximate prices, AliExpress welcome-deal note, loading bar, "prices updated X ago".

> **To resume in a new Claude Code session:** "Continue the Owis_Find_Deal_Engine project from `Owis_Find_Deal_Engine/STATUS.md` on branch `claude/cloud-vs-local-7xi988`. Start phase 3, step 12."

Other documents: `STATUS.md` (handoff + owner test steps), `PLAN.md` (design + all steps), `server/README.md` (reference), `docs/current_flow.pdf`, `next_plan.pdf`, `architecture.pdf` (older snapshots). Regenerate this guide with `python3 docs/build_service_guide_pdf.py`.